

Supervised learning competition

AUTHORS

Jack Hewett
Stefan Houkes
Bilal Mokhtar
Anthony Kamau
Lars van den Bosch

PUBLISHED

October 21, 2025

We first beginn by loading all the required packages which are the following:

▸ [Code](#)

Data description

We then load our training and test data

▸ [Code](#)

We then checked for missing values in our training data

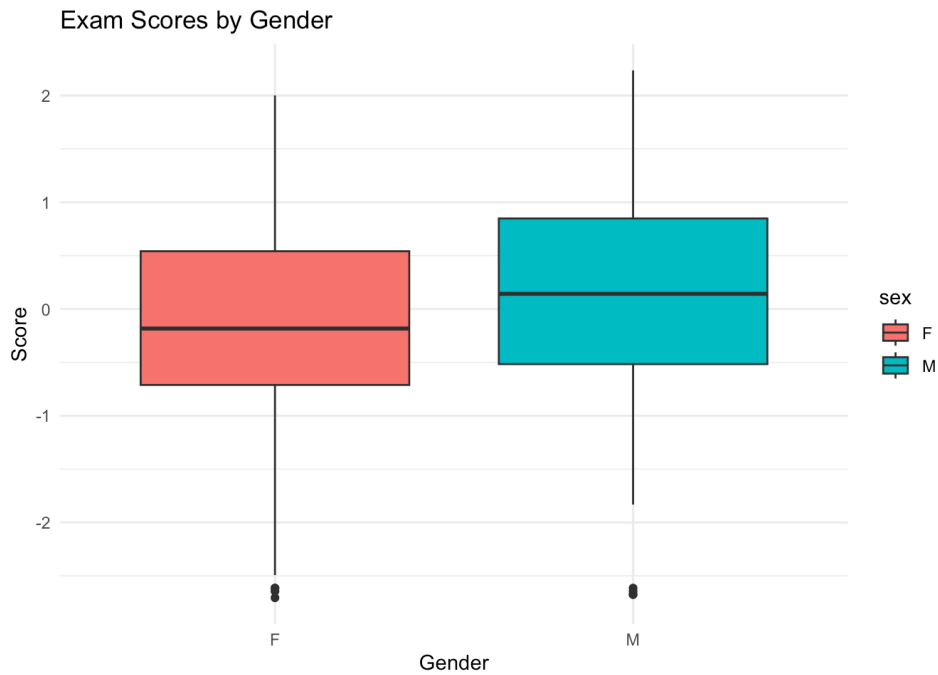
▸ [Code](#)

In the first figure we visualized the distribution of the normalized score. The data looks normally distributed with some outliers. We see a small increase at the low scores. This could be because some students did not study or did not show so they received the lowest score. When we compare male and female students we see that male students receive a higher score but it is not yet proven to be significant. when we compare study time, we only see a small impact of study time on the normalized score.

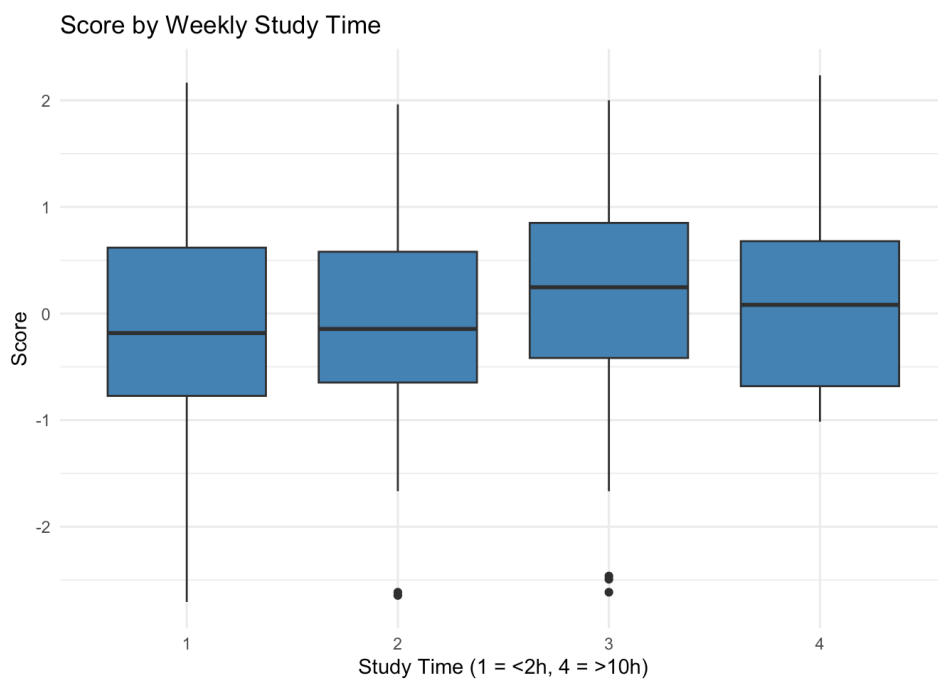
▸ [Code](#)



► Code



► Code



► Code

Model description

We used the following models:

Liner regression

We used linear regression because it provides a simple and interpretable baseline model that assumes a straight-line relationship between

predictors and the outcome. By starting with linear regression, we easily understand how each variable contributes to the prediction. It gives us a clear benchmark to compare against more complex models.

Lasso Regression

We used Lasso regression because it is a linear model that not only fits relationships between predictors and the target but also performs automatic feature selection. By applying an L1 penalty, Lasso shrinks less important coefficients exactly to zero, leaving us with a simpler, more interpretable model. This is especially useful when we have many predictors, some of which may be irrelevant or highly correlated, since Lasso helps us focus on the most influential variables while reducing overfitting.

K Nearest Neighbor (KNN)

We used k-Nearest Neighbors (kNN) because it is a simple, non-parametric method that makes predictions based on similarity. Instead of assuming a specific functional form between predictors and the target, kNN looks at the closest training examples to a new observation and bases its prediction on them. This makes it flexible and intuitive, particularly when the relationship between features and the outcome is complex or nonlinear. It also provides a useful contrast to regression-based models, since it relies purely on local patterns in the data.

Random Forest

We used a Random Forest because it is a powerful ensemble method that combines many decision trees to improve predictive accuracy and robustness. Each tree is trained on a random subset of the data and features, and their predictions are averaged, which reduces variance and guards against overfitting. Random Forests can capture complex nonlinear relationships and interactions between variables without requiring much preprocessing, making them a strong baseline model in many applied machine learning tasks.

Data transformation and pre-processing

First, we identified which variables in the dataset are categorical, such as sex, school, address and others. We then explicitly converted them into factors in both the training and test datasets. This step is important because categorical data needs to be treated differently from numeric data during modeling, ensuring that the algorithms recognize them as categories rather than continuous numbers.

► Code

Next, we made sure that the target variable, score, was stored as a numeric type. This guarantees that the models interpret it correctly as a continuous outcome to be predicted, rather than as a factor or character string. After this, we checked the structure of the training data to confirm that all variables were in the correct format before proceeding.

► Code

```
tibble [316 × 31] (S3: tbl_df/tbl/data.frame)
 $ school   : Factor w/ 2 levels "GP","MS": 1 1 1 1 1 1 1 1 1 1
1 ...
 $ sex      : Factor w/ 2 levels "F","M": 1 1 2 2 1 1 2 1 1 2
...
 $ age      : num [1:316] 18 17 17 18 16 15 15 15 16 15 ...
 $ address  : Factor w/ 2 levels "U","R": 1 1 1 2 1 2 1 1 1 1
...
 $ famsize  : Factor w/ 2 levels "GT3","LE3": 1 1 1 1 1 2 2 1
1 1 ...
 $ Pstatus  : Factor w/ 2 levels "A","T": 2 2 2 2 2 2 2 2 2 2
...
 $ Medu     : num [1:316] 3 3 2 4 1 2 3 4 3 2 ...
 $ Fedu     : num [1:316] 2 4 3 3 3 2 3 4 3 3 ...
 $ Mjob     : Factor w/ 5 levels "at_home","health",...: 3 4 3
5 1 2 3 4 3 3 ...
 $ Fjob     : Factor w/ 5 levels "teacher","other",...: 3 2 2 3
3 3 2 5 2 2 ...
 $ reason   : Factor w/ 4 levels "course","other",...: 2 1 1 1
3 4 1 1 3 3 ...
 $ guardian : Factor w/ 3 levels "mother","father",...: 1 1 2 1
1 1 1 1 1 1 ...
 $ traveltime: num [1:316] 1 1 2 1 1 2 1 1 1 1 ...
 $ studytime : num [1:316] 3 3 1 3 2 2 2 3 3 3 ...
 $ failures  : num [1:316] 0 0 0 0 3 0 0 0 0 0 ...
 $ schoolsup : Factor w/ 2 levels "yes","no": 2 2 2 2 2 1 2 2 2
1 ...
 $ famsup    : Factor w/ 2 levels "no","yes": 1 1 1 1 1 2 1 2 2
1 ...
 $ paid      : Factor w/ 2 levels "no","yes": 1 1 1 1 1 2 1 1 2
2 ...
 $ activities: Factor w/ 2 levels "no","yes": 1 1 1 1 2 1 2 2 1
1 ...
 $ nursery   : Factor w/ 2 levels "yes","no": 1 1 1 1 2 1 2 1 1
2 ...
 $ higher    : Factor w/ 2 levels "yes","no": 1 1 1 1 1 1 1 1 1
1 ...
 $ internet  : Factor w/ 2 levels "no","yes": 2 2 2 2 2 2 2 2 2
2 ...
 $ romantic  : Factor w/ 2 levels "no","yes": 2 1 1 2 2 1 1 2 2
1 ...
 $ famrel    : num [1:316] 5 4 5 5 4 4 5 4 4 5 ...
 $ freetime  : num [1:316] 4 4 2 3 3 1 3 3 3 3 ...
 $ goout     : num [1:316] 3 5 2 2 5 3 2 3 3 2 ...
 $ Dalc      : num [1:316] 2 1 1 1 1 1 1 1 1 1 ...
 $ Walc      : num [1:316] 3 3 1 2 1 3 1 1 3 2 ...
 $ health    : num [1:316] 1 5 2 4 3 4 2 5 4 5 ...
```

```
$ absences : num [1:316] 7 16 4 9 0 2 0 0 0 4 ...  
$ score    : num [1:316] 0.648 1.169 0.384 1.207 -1.215 ...
```

We then identified which predictors were numeric, excluding the target variable. These numeric predictors were standardized using scaling, which transforms them to have a mean of zero and a standard deviation of one. Scaling ensures that predictors measured on different scales, such as study time and absences, contribute fairly to the model without one dominating due to its larger numeric range.

► Code

Finally, we took the scaling parameters, such as the means and standard deviations, from the training data and used them to transform the test data in the same way. This approach helps prevent data leakage by making sure the test set is scaled just like the training set, using only information from the training data. As a result, we have two clean and well-prepared datasets: one for training and one for testing. Both are ready for reliable model building and evaluation.

► Code

We split the predictors from the target so the model can learn from one set and predict the other. All input features except for the score go into one object, while the score values are stored separately. We also keep a full copy of the scaled training data, which gives us flexibility for modeling and a complete dataset to refer back to.

► Code

Model Comparison

The first step in comparing our models i.e Random Forest, Lasso, k-Nearest Neighbours (kNN) and Linear Regression is to test them under identical conditions. This entails applying cross-validation, using the same training and test splits and evaluating performance using a standardised metric like **MSE** which can be derived from **RMSE**. By standardising the assessment procedure we make sure that variations in outcomes are caused by the models and not by inconsistent testing.

Every model has unique advantages. linear regression is the most straightforward and less complex model which offers a distinct performance baseline. Lasso expands on our linear model by incorporating regularisation, which can enhance prediction and automatically identify significant features. Random Forest models intricate nonlinear relationships and interactions and kNN finds local patterns in the data. By contrasting them, we determine whether a straightforward, understandable model is adequate or if more intricate approaches give significant gains.

Cross Validation

We used five-fold cross-validation i.e training on four parts and testing on the remaining one.

▸ Code

Linear regression

▸ Code

Linear Regression

316 samples

30 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 253, 253, 252, 254, 252

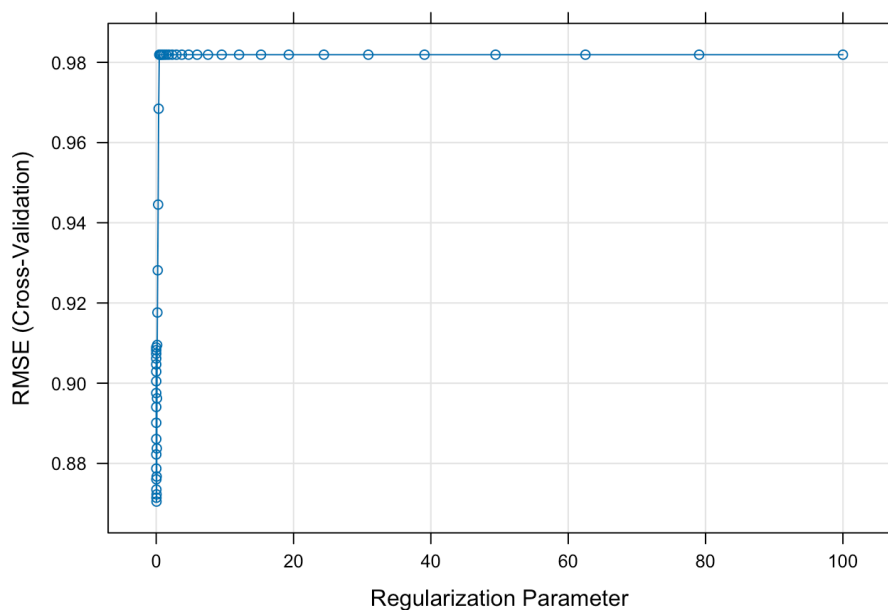
Resampling results:

RMSE	Rsquared	MAE
0.9115015	0.1854684	0.7365162

Tuning parameter 'intercept' was held constant at a value of TRUE

Lasso

▸ Code



▸ Code

glmnet

316 samples

30 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)
 Summary of sample sizes: 253, 253, 252, 254, 252
 Resampling results across tuning parameters:

lambda	RMSE	Rsquared	MAE
1.000000e-03	0.9089118	0.1865706	0.7341625
1.264855e-03	0.9082421	0.1868712	0.7335314
1.599859e-03	0.9073533	0.1873199	0.7326913
2.023590e-03	0.9061601	0.1879895	0.7315548
2.559548e-03	0.9046875	0.1888382	0.7301278
3.237458e-03	0.9028868	0.1899094	0.7283388
4.094915e-03	0.9004864	0.1915605	0.7260286
5.179475e-03	0.8975342	0.1937769	0.7233308
6.551286e-03	0.8940630	0.1964511	0.7201708
8.286428e-03	0.8901240	0.1996489	0.7163242
1.048113e-02	0.8860818	0.2030473	0.7124446
1.325711e-02	0.8822343	0.2063945	0.7097822
1.676833e-02	0.8787142	0.2095547	0.7067476
2.120951e-02	0.8760356	0.2118599	0.7039853
2.682696e-02	0.8734798	0.2151866	0.7005731
3.393222e-02	0.8714308	0.2193967	0.6968483
4.291934e-02	0.8704798	0.2241799	0.6938380
5.428675e-02	0.8722646	0.2268667	0.6930978
6.866488e-02	0.8767299	0.2247048	0.6963511
8.685114e-02	0.8837400	0.2169241	0.6994075
1.098541e-01	0.8962209	0.1944156	0.7063775
1.389495e-01	0.9095311	0.1714667	0.7154389
1.757511e-01	0.9176200	0.1684808	0.7208150
2.222996e-01	0.9281488	0.1684936	0.7308038
2.811769e-01	0.9445345	0.1684887	0.7474606
3.556480e-01	0.9684384	0.1422391	0.7706252
4.498433e-01	0.9819098	NaN	0.7834553
5.689866e-01	0.9819098	NaN	0.7834553
7.196857e-01	0.9819098	NaN	0.7834553
9.102982e-01	0.9819098	NaN	0.7834553
1.151395e+00	0.9819098	NaN	0.7834553
1.456348e+00	0.9819098	NaN	0.7834553
1.842070e+00	0.9819098	NaN	0.7834553
2.329952e+00	0.9819098	NaN	0.7834553
2.947052e+00	0.9819098	NaN	0.7834553
3.727594e+00	0.9819098	NaN	0.7834553
4.714866e+00	0.9819098	NaN	0.7834553
5.963623e+00	0.9819098	NaN	0.7834553
7.543120e+00	0.9819098	NaN	0.7834553
9.540955e+00	0.9819098	NaN	0.7834553
1.206793e+01	0.9819098	NaN	0.7834553
1.526418e+01	0.9819098	NaN	0.7834553
1.930698e+01	0.9819098	NaN	0.7834553
2.442053e+01	0.9819098	NaN	0.7834553
3.088844e+01	0.9819098	NaN	0.7834553
3.906940e+01	0.9819098	NaN	0.7834553
4.941713e+01	0.9819098	NaN	0.7834553
6.250552e+01	0.9819098	NaN	0.7834553
7.906043e+01	0.9819098	NaN	0.7834553
1.000000e+02	0.9819098	NaN	0.7834553

Tuning parameter 'alpha' was held constant at a value of 1

RMSE was used to select the optimal model using the smallest value.
The final values used for the model were alpha = 1 and lambda = 0.04291934.

K-nearest neighbor

► Code

k-Nearest Neighbors

316 samples
30 predictor

Pre-processing: centered (39), scaled (39)
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 253, 253, 252, 254, 252
Resampling results across tuning parameters:

k	RMSE	Rsquared	MAE
1	1.3265477	0.01854216	1.0118978
2	1.1300521	0.03026473	0.8788399
3	1.0388791	0.05382995	0.8180851
4	1.0078448	0.06264119	0.7910994
5	0.9914077	0.06561219	0.7689407
6	0.9853730	0.06363430	0.7596850
7	0.9707255	0.07614115	0.7505120
8	0.9625406	0.08483399	0.7476576
9	0.9575704	0.08574506	0.7402255
10	0.9465880	0.10291282	0.7367933
11	0.9548974	0.08957829	0.7461586
12	0.9438296	0.10541559	0.7379619
13	0.9417551	0.10619556	0.7364561
14	0.9435463	0.10207452	0.7343501
15	0.9377741	0.10693216	0.7284412
16	0.9403732	0.10587458	0.7352669
17	0.9409386	0.10052903	0.7370098
18	0.9427888	0.09528061	0.7391425
19	0.9453062	0.09006224	0.7420627
20	0.9445106	0.09157291	0.7425107

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was k = 15.

Random Forest

► Code

Random Forest

316 samples
30 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 253, 253, 252, 254, 252

Resampling results across tuning parameters:

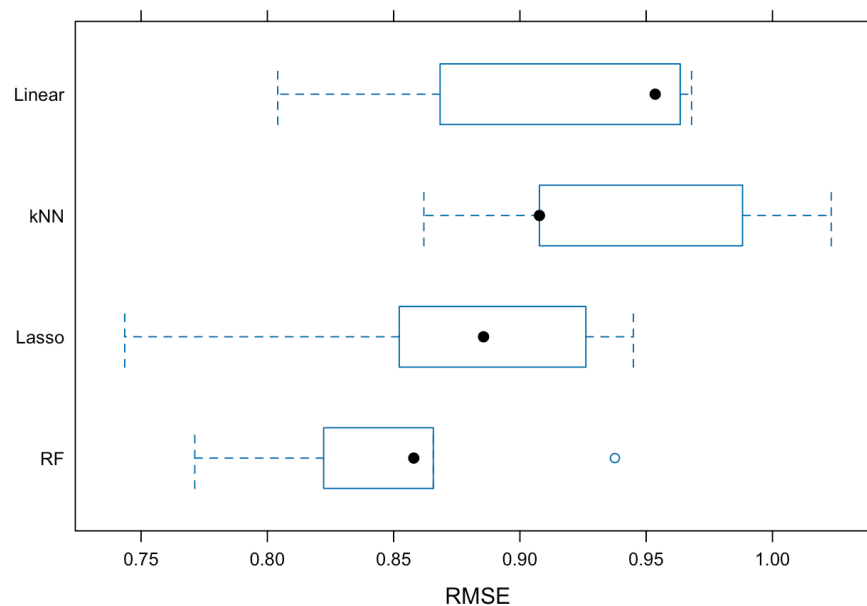
mtry	RMSE	Rsquared	MAE
2	0.9015394	0.2210352	0.7102386
11	0.8570955	0.2537818	0.6646732
20	0.8509733	0.2601467	0.6642405
29	0.8514889	0.2595387	0.6646337
39	0.8565131	0.2533983	0.6698047

RMSE was used to select the optimal model using the smallest value.

The final value used for the model was mtry = 20.

Chosen model

► Code



To find **mse** we just square **rmse**. The **mse** for the models are:

► Code

Linear	Lasso	RF	kNN
0.8308350	0.7577351	0.7241556	0.8794202

► Code

model	performance	Reason
Linear regression	0.83	Simplicity
Lasso regression	0.76	Regularization
K-nearest neighbor	0.88	Locality
Random forest	0.72	Ensemble

Team member contributions

Write down what each team member contributed to the project.

- **Jack Hewett** - knn, data data description, model comparison, lasso
- **Stefan Houkes** - linear regression, data preparation, model comparison, lasso
- **Bilal Mokhtar** - knn, model comparison, data description, lasso
- **Anthony Kamau** - random forest, data description, model comparison, lasso
- **Lars van den Bosch** - linear regression, data preparation, model comparison, lasso